第 6 章 获取模型：渠道、订阅、API 与成本

> 本章目标：分清"聊天产品 / API / 本地模型"三种用法，知道在国内怎样合法地用到模型，能自己动手估一次课程的成本，并守住 API key 这条安全底线。

> 要点：决定你用哪种方式接模型的因素只有三个——你愿意花多少钱、数据能不能离开你的电脑、以及是否需要把模型嵌进程序里。

6.1 三层概念：产品、API、本地模型

初学者最容易混淆的是"我在网页上聊天"和"我的程序在调用模型"——这其实是两种完全不同的东西。

第一层：产品（网页 / App 聊天）
你在网页或 App 里打字、模型回话。你买的是"成品软件"：界面、历史记录、文件上传、联网搜索都做好了。计费方式通常是月付订阅或免费额度，你不需要写一行代码。

第二层：API（应用程序接口）
API 是"给程序用的插座"。你的代码发一段请求过去，模型返回一段结果。它没有界面，按用量计费，可以嵌进任何程序：课堂演示系统、自动批改脚本、你们做的机器人控制台。
类比：产品是"去餐厅吃饭"，API 是"买食材回家自己做"——后者更灵活、也可以更便宜，但锅碗瓢盆得你自己备。

第三层：本地模型（自己运行）
把模型权重下载到自己的电脑或服务器上运行，不经过任何外部服务器。需要足够的显存或内存，速度取决于你的硬件。

维度	网页 / App 产品	API 接口	本地模型
计费方式	月付订阅或免费额度，有频率限制	按 token 用量计费，用多少算多少	一次性硬件投入 + 电费，无调用费
数据去留	内容会上传到服务商	内容会通过服务商接口传输	数据不出本机
速度	取决于服务商排队情况	通常较稳定，高峰期可能变慢	取决于显存与模型大小
能力	与官方产品同步更新	与产品可能不完全同步，取决于版本	通常弱于同代云端大模型
典型用户	教师备课、学生日常问答、写作	想把模型接进自己程序的开发者	离线教学、涉密笔记、重度反复调用

这三层不是替代关系。一个现实的组合是：备课时用网页产品，做课堂演示程序时用 API，演示现场断网时切到本地小模型兜底。

6.2 国内模型：合规、直接、门槛低

国内主流模型服务都可以用手机号注册，网页版和 API 在同一个账号体系下，支付走国内常见渠道。对大陆的课程来说，这是最省事的路径，也不存在账号合规问题。

常见的几个服务（只列名称，不涉及版本号）：

服务	提供方	常见形态	说明
通义千问（Qwen）	阿里云	网页 / App / API	生态完整，文档偏工程向

服务	提供方	常见形态	说明
DeepSeek	深度求索	网页 / App / API	推理类任务口碑较好，价格档位常被拿来对比
豆包 / Seed	字节跳动	App / 网页 / API	面向普通用户的交互做得比较顺手
智谱 GLM	智谱 AI	网页 / API	较早开放 API 的平台之一
Kimi（Moonshot）	月之暗面	网页 / App / API	长文档阅读场景用得较多
文心一言	百度	网页 / App / API	与百度生态绑定较紧

选择建议：先注册两三家，各用同一道题测一下。**不要只看"谁更强"，要看谁在你的具体任务上更稳。**

6.3 海外模型：能力强，但门槛在账号和支付

OpenAI、Anthropic、Google、xAI 等海外服务，能力通常处于第一梯队，但对大陆的教师和学生而言，门槛不在技术，而在两件事：**支付渠道**和**账号资格**。多数服务需要境外发行的银行卡，有的还需要境外手机号验证。

常见的支付路径（作为商业现象介绍，不是操作指南）：

- **官方直连**：Visa / Mastercard 信用卡，Google Play / Google Pay，PayPal，Apple Pay。最稳，但需要一张能用的境外卡。
- **代充**：由第三方代你付款。风险是对方跑路、账号被关联封禁，以及你的账号信息经手了陌生人。
- **拼车 / 共享订阅**：多人共用一个账号分摊订阅费。风险是账号一旦被平台判定异常就会被封，且同组人能看到彼此的对话历史。

合规边界（本章只说一次）：请遵守各服务的用户协议和所在地法律法规，本课程不教授绕过实名认证、地区限制或服务条款的方法。 网络接入与账号准备属于课前事项，由教师或学生按合法渠道自行处理；学校或企业通常已有合规的统一网关或内部模型服务，先问一句往往比自己折腾省事。

6.4 订阅与 API：两种完全不同的账

订阅制是"包月"，API 是"按量"。两者的差别不只是价格，而是**成本的可预测性**。

	订阅（包月）	API（按量）
计费	固定月费	按 token 数计费
限制	有频率/额度上限，超出后限速或降级	没有"月限额"，但有余额
风险	用不满就浪费；用超了被限流	代码写错可能一夜烧掉预算
适合	人在聊天框里用	程序自动调用、批量任务

API 计费的基本结构：输入 token 和输出 token 分开计价，输出通常更贵。部分服务对重复出现的上下文（比如固定的系统提示词）提供缓存折扣，命中缓存的部分单价更低。这里不列任何具体数字，因为价格会变——**一切以各家官网的价目页为准**。

估算方法（带算式的例子）

不要背价格，要会套公式。

```
总费用 = 输入 token 总数 ÷ 1,000,000 × 输入单价
       + 输出 token 总数 ÷ 1,000,000 × 输出单价
```

假设一节课的设计是：30 名学生，每人 20 轮对话，平均每轮输入 1,000 token、输出 500 token。

```
输入 token 总数 = 30 × 20 × 1,000 = 600,000
输出 token 总数 = 30 × 20 ×   500 = 300,000

费用 = 600,000 ÷ 1,000,000 × 输入单价
     + 300,000 ÷ 1,000,000 × 输出单价
     = 0.6 × 输入单价 + 0.3 × 输出单价
```

把官网的两个单价代进去，就是这节课的模型成本。若要算一学期，乘以课时数即可。

两个坑要提前记住：**多轮对话的输入是累积的**（第 10 轮往往要把前面九轮一起发过去，实际输入远大于 1,000，最省事的办法是直接看服务商后台的用量面板）；**调试阶段比正式使用更贵**，估预算时要留余量。

6.5 中转站：省事，但要清楚代价

"中转站"是第三方搭建的转发服务：你拿着它的 key，请求经它转给上游模型。它存在的原因很实际——上游支付门槛高、注册麻烦，而中转站收国内支付方式、随开随用。

它的风险同样实际：

- **数据经过第三方**：你的提示词和回复内容都会落到中转站的服务器上。
- **跑路**：预付费模式，服务方随时可能消失，余额归零。
- **模型替换**：你付的是某个模型的钱，对方实际转发给一个更便宜的模型，你从输出来看不出区别。
- **key 泄露**：中转站同时保管大量用户的凭据，一旦泄露波及面很大。

怎么降低风险：

- **先小后大**：先充最小额度，用固定题目测几十次，再决定是否投入。
- **看延迟与错误率**：长期异常的响应速度、莫名其妙的 5xx 错误、与官方明显不同的限流表现，都是信号。
- **不要相信模型"自报家门"**：问它"你是谁"，它回答什么由提示词决定，不能当作验证手段。

- **私有数据一律走官方 API 或本地模型**：学生的作业、个人信息、未公开的代码，不要经手第三方转发。

6.6 本地模型：成本换了个地方

本地跑模型的成本不在"调用费"，而在硬件、电费和时间：模型越大需要的显存/内存越多，量化（把权重压成更低精度）能显著缩小占用，代价是精度略降；长时间满载的电费和噪音是真实开销；生成速度则取决于硬件。

它的优势在另外三件事：**数据不出机器、断网可用、反复调用边际成本接近零**。

什么时候值得本地化：

场景	建议
机房断网、教室无外网	本地模型，选能跑起来的最小档位
处理学生成绩、家庭信息等敏感数据	本地模型优先
每天反复跑几千次的小任务（分类、改写、格式转换）	本地模型长期更划算
需要最强推理和最新知识	云端 API

6.7 安全实践清单

API key 就是钱。任何人拿到它，就能以你的名义消费。

- [ ] key 只放在服务端或本地环境变量里，**绝不写进前端代码**（浏览器的"查看源代码"就能看到）。
- [ ] 建 `.env` 文件存 key，把 `.env` 加进 `.gitignore`，先提交 `.env.example` 模板。
- [ ] 提交前搜一遍仓库里有没有 `sk-` 开头的字符串和硬编码的 token。
- [ ] 在服务商后台设置**用量上限与告警**，别让程序 bug 变成账单。

- [ ] 给每个项目单独申请 key，便于定位和单独吊销。
- [ ] 一旦发现泄露（截图、录屏、误推送到公开仓库），**立刻去后台吊销并重新生成**，不要抱侥幸心理。
- [ ] 群里分享代码时检查是否带着 key；演示投屏时不要切到含 key 的终端窗口。

> 要点：key 泄露后的第一动作是吊销，不是解释。重新生成 key 只要一分钟，追查是怎么泄露的可能要一整天。

课后题

1. 用本章的公式，估算"40 名学生、每人 15 轮、平均输入 1,200 token、输出 400 token"的一节课成本，写出完整算式（单价用官网现价，注明查询日期）。
2. 把"网页产品 / API / 本地模型"三层各举一个你自己真实的使用场景，并说明为什么那一层最合适。
3. 有人说："中转站便宜又方便，为什么不用？"请用本章的四个风险点，写出你的三点回应。
4. 检查你（或你小组）的项目仓库：有没有把 key 写进代码、有没有 `.gitignore`、有没有开启用量告警？列出发现的问题。
5. 开放题：如果学校机房完全断网，你会怎么设计一套"AI 辅助教学"的课堂方案？至少写出两个技术上的替代路径。